

1. ZADATAK

PRII – G2

Izvršiti definiciju funkcija na način koji odgovara opisu (komentarima) datom neposredno uz pozive ili nazive funkcija. Možete dati komentar na bilo koju liniju code-a koju smatrate da bi trebalo unaprijediti ili da će eventualno uzrokovati grešku prilikom kompajliranja. Također, možete dodati dodatne funkcije ili metode koje će vam olakšati implementaciju programa.

```
#include <iostream>
using namespace std;

string crt = "\n-----\n";

string PORUKA_TELEFON = crt +
"TELEFONE ISKLJUCITE I ODLOZITE U TORBU, DZEP ILI DRUGU LOKACIJU VAN
DOHVATA.\n"
"CESTO SE NA TELEFONIMA (PRO)NALAZE PROGRAMSKI KODOVI KOJI MOGU BITI
ISKORISTENI ZA\n"
"RJESAVANJE ISPITNOG ZADATKA, STO CE, U SLUCAJU PRONALASKA, BITI
SANKCIONISANO." + crt;

string PORUKA_ISPIT = crt +
"0. PROVJERITE DA LI ZADACI PRIPADAJU VASOJ GRUPI (G1/G2)\n"
"1. SVE KLASA SA DINAMICKOM ALOKACIJOM MORAJU IMATI ISPRAVAN
DESTRUKTOR\n"
"2. IZOSTAVLJANJE DESTRUKTORA ILI NJEGOVIH DIJELOVA BIT CE OZNACENO
KAO TM\n"
"3. ATRIBUTI, METODE I PARAMETRI MORAJU BITI IDENTICNI ONIMA U TESTNOJ
MAIN FUNKCIJI,\n"
"    OSIM AKO POSTOJI JASNO OPISAN RAZLOG ZA MODIFIKACIJU\n"
"4. IZUZETKE BACAJTE SAMO TAMO GDJE JE IZRICITO NAGLASENO\n"
"5. SVE METODE KOJE SE POZIVAJU U MAIN-U MORAJU POSTOJATI.\n"
"    AKO NEMATE ZELJENU IMPLEMENTACIJU, OSTAVITE PRAZNO TIJELO ILI
VRATITE DEFAULT VRIJEDNOST\n"
"6. U MAIN FUNKCIJI MOZETE DODAVATI TESTNE PODATKE I POZIVE PO
VLASTITOM IZBORU\n"
"7. TESTIRAJTE PROGRAM U OBA MODA (F5 i Ctrl+F5)" + crt;

char* AlocirajTekst(const char* tekst) {
    if (!tekst) return nullptr;
    size_t vel = strlen(tekst) + 1;
    char* temp = new char[vel];
    strcpy_s(temp, vel, tekst);
    return temp;
}

string GenerisiClanskiBroj(const char* imePrezime, int redniBroj);
bool ValidirajClanskiBroj(const string& clanskiBroj);

enum Zanr { ROMAN, STRUCNA_LITERATURA, BIOGRAFIJA, POEZIJA,
DJECIJA_KNJIGA };
const char* ZanrNazivi[] = { "ROMAN", "STRUCNA LITERATURA",
"BIOGRAFIJA", "POEZIJA", "DJECIJA KNJIGA" };

template<class T1, class T2, int max>
class Kolekcija {
```

```

    T1* _prvi;
    T2* _drugi;
    int _trenutno;
public:
    Kolekcija() : _prvi(nullptr), _drugi(nullptr), _trenutno(0) {}
    int GetTrenutno() const { return _trenutno; }
    T1& GetPrvi(int indeks) { return _prvi[indeks]; }
    T2& GetDrugi(int indeks) { return _drugi[indeks]; }
    T1& operator[](int indeks) { return _prvi[indeks]; }
    friend ostream& operator<<(ostream& COUT, Kolekcija& obj) {
        for (int i = 0; i < obj.GetTrenutno(); i++)
            COUT << obj.GetPrvi(i) << " " << obj.GetDrugi(i) << endl;
        return COUT;
    }
    ~Kolekcija() {
        delete[] _prvi; _prvi = nullptr;
        delete[] _drugi; _drugi = nullptr;
    }
};

class Datum {
    int* _godina, * _mjesec, * _dan;
public:
    Datum(int dan = 1, int mjesec = 1, int godina = 2000) {
        _godina = new int(godina);
        _mjesec = new int(mjesec);
        _dan = new int(dan);
    }
    ~Datum() {
        delete _godina; delete _mjesec; delete _dan;
    }
};

class Posudba {
    char* _naslov;
    Zanr _zanr;
    Datum _datumPosudbe;
    int _brojDana;
public:
    Posudba(const char* naslov, Zanr zanr, Datum datumPosudbe, int
    brojDana)
        : _zanr(zanr), _datumPosudbe(datumPosudbe),
        _brojDana(brojDana) {
        _naslov = AlocirajTekst(naslov);
    }
    ~Posudba() { delete[] _naslov; _naslov = nullptr; }
    const char* GetNaslov() const { return _naslov; }
    Zanr GetZanr() const { return _zanr; }
    Datum& GetDatumPosudbe() { return _datumPosudbe; }
    int GetBrojDana() const { return _brojDana; }
};

class ClanBiblioteke {
    static int _id;
    char* _clanskiBroj;
    char* _imePrezime;
    vector<Posudba> _posudbe;
public:

```

```

    ClanBiblioteke(const char* imePrezime = "") {
        _imePrezime = AlocirajTekst(imePrezime);
        _clanskiBroj = AlocirajTekst(GenerisiClanskiBroj(imePrezime,
_id).c_str());
        _id++;
    }
    ~ClanBiblioteke() {
        delete[] _clanskiBroj; _clanskiBroj = nullptr;
        delete[] _imePrezime; _imePrezime = nullptr;
    }
    const char* GetClanskiBroj() const { return _clanskiBroj; }
    const char* GetImePrezime() const { return _imePrezime; }
    vector<Posudba>& GetPosudbe() { return _posudbe; }
    friend ostream& operator<<(ostream& COUT, ClanBiblioteke& obj) {
        COUT << obj._imePrezime << " [" << obj._clanskiBroj << "]" <<
endl;
        for (auto& posudba : obj._posudbe)
            //ToString metoda vraća podatke o posudbi u formatu
prikazanom u nastavku.
            //voditi računa o prikazu jednocifrenih vrijednosti
datuma (npr. 5 -> 05).
            //05.10.2026 Tvrdjava ROMAN 14 dana
            COUT << " - " << posudba.ToString() << endl;
        return COUT;
    }
};
int ClanBiblioteke::_id = 1;

class Biblioteka {
    char* _naziv;
    vector<ClanBiblioteke> _clanovi;
public:
    Biblioteka(const char* naziv) { _naziv = AlocirajTekst(naziv); }
    ~Biblioteka() { delete[] _naziv; _naziv = nullptr; }
    Biblioteka(const Biblioteka& obj) {
        _naziv = AlocirajTekst(obj._naziv);
        _clanovi = obj._clanovi;
    }
    const char* GetNaziv() const { return _naziv; }
    vector<ClanBiblioteke>& GetClanovi() { return _clanovi; }
};

const char* GetOdgovorNaPrvoPitanje() {
    cout << "Pitanje -> Pojasnite kako tip nasljeđivanja (public,
protected, private) utiče na dostupnost članova bazne klase u
izvedenoj klasi.          Sta se desava sa public i protected članovima
bazne klase ako se nasljeđivanje izvrši kao protected ili
private?.\n";
    return "Odgovor -> OVDJE UNESITE VAS ODGOVOR";
}

const char* GetOdgovorNaDrugoPitanje() {
    cout << "Pitanje -> Pojasnite ulogu i značaj move konstruktora,
kada se poziva i po čemu se razlikuje od konstruktora kopije.\n";
    return "Odgovor -> OVDJE UNESITE VAS ODGOVOR";
}

int main() {

```

```
cout << PORUKA_TELEFON; cin.get();
cout << PORUKA_ISPIT; cin.get(); system("cls");
cout << GetOdgovorNaPrvoPitanje() << crt;
cin.get();
cout << GetOdgovorNaDrugoPitanje() << crt;
cin.get();

//funkcija generise clanski broj na osnovu imena i prezimena,
rednog broja i trenutne godine.
//clanski broj je u formatu GGGG/IN-BBB, gdje GGGG predstavlja
trenutnu godinu, IN inicijale,
//a BBB redni broj clana popunjen nulama na tri mjesta.
//funkciju koristiti u konstruktoru klase ClanBiblioteke za
inicijalizaciju atributa _clanskiBroj.
if (GenerisiClanskiBroj("Amina Buric", 3) == "2026/AB-003")
    cout << "Clanski broj OK" << crt;
if (GenerisiClanskiBroj("Amar Macic", 15) == "2026/AM-015")
    cout << "Clanski broj OK" << crt;
if (GenerisiClanskiBroj("Maid Ramic", 156) == "2026/MR-156")
    cout << "Clanski broj OK" << crt;

//ValidirajClanskiBroj koristeci regex provjerava format definisan
u prethodnom dijelu zadatka.
if (ValidirajClanskiBroj("2026/AB-003"))
    cout << "CLANSKI BROJ VALIDAN" << crt;
if (!ValidirajClanskiBroj("2026/Ab-003")) &&
!ValidirajClanskiBroj("26/AB-003") &&
!ValidirajClanskiBroj("2026-AB/003"))
    cout << "CLANSKI BROJ NIJE VALIDAN" << crt;

Kolekcija<int, string, 20> inventar;
for (int i = 0; i < 8; i++)
    inventar.Dodaj(i, "Knjiga_" + to_string(i));
cout << inventar << crt;

//DodajNaPoziciju dodaje novi par na lokaciju definisanu prvim
parametrom. metoda vraca trenutno stanje kolekcije.
//u slucaju popunjene kolekcije ili neispravne lokacije potrebno
je baciti izuzetak (za potrebe testiranja mozete dodati try catch
blok).
Kolekcija<int, string, 20> prosireniInventar =
inventar.DodajNaPoziciju(2, 99, "Posebno izdanje");
cout << prosireniInventar << crt;

//UkloniRaspon uklanja broj elemenata definisan drugim parametrom,
pocevsi od lokacije
//definisane prvim parametrom (ukljucujuci tu lokaciju). metoda
vraca pokazivac na novu kolekciju sa uklonjenim
//elementima, a pozivalac je odgovoran za njenu dealokaciju.
Kolekcija<int, string, 20>* uklonjeneKnjige =
prosireniInventar.UkloniRaspon(3, 2);
cout << "Uklonjeni elementi:" << crt << *uklonjeneKnjige;
cout << "Preostali elementi:" << crt << prosireniInventar;
delete uklonjeneKnjige;

try {
    //za neispravan raspon potrebno je baciti izuzetak
```

```
        inventar.UkloniRaspon(6, 5);
    }
    catch (exception& e) {
        cout << "Exception: " << e.what() << crt;
    }

    Datum datum1(5, 10, 2026), datum2(7, 10, 2026), datum3(10, 10,
2026);
    Posudba tvrdjava("Tvrdjava", ROMAN, datum1, 14);
    Posudba cpp("Programiranje u C++", STRUCNA_LITERATURA, datum1,
30);
    Posudba tesla("Nikola Tesla", BIOGRAFIJA, datum2, 20);
    Posudba pjesme("Izabrane pjesme", POEZIJA, datum3, 10);

    //ToString metoda vraca podatke o posudbi u formatu prikazanom u
nastavku.
    //voditi racuna o prikazu jednocifrenih vrijednosti datuma (npr. 5
-> 05).
    cout << tvrdjava.ToString() << crt;
    //05.10.2026 Tvrdjava ROMAN 14 dana

    ClanBiblioteke amina("Amina Buric"), goran("Goran Skondric"),
berun("Berun Agic");

    //DodajPosudbu dodaje posudbu ako clan nema ranije zaduzenu knjigu
istog naslova,
    //ako trenutno ima manje od tri posudbe i ako je broj dana posudbe
u rasponu od 1 do 30.
    //metoda vraca true ako je posudba dodana, u suprotnom vraca
false.
    if (amina.DodajPosudbu(tvrdjava))
        cout << "Posudba dodana" << crt;
    if (!amina.DodajPosudbu(tvrdjava))
        cout << "Posudba nije dodana - knjiga je vec zaduzena" <<
crt;
    amina.DodajPosudbu(cpp);
    amina.DodajPosudbu(tesla);
    if (!amina.DodajPosudbu(pjesme))
        cout << "Posudba nije dodana - dostignut maksimalan broj
posudbi" << crt;

    //RazduziKnjigu uklanja posudbu na osnovu naslova knjige i vraca
true ako je pronadjena
    //i uklonjena. ukoliko knjiga nije pronadjena metoda vraca false.
    if (amina.RazduziKnjigu("Tvrdjava"))
        cout << "Knjiga razduzena" << crt;
    if (!amina.RazduziKnjigu("Nepostojeca knjiga"))
        cout << "Knjiga nije pronadjena" << crt;

    Biblioteka          gradska("Gradska          biblioteka"),
univerzitetska("Univerzitetska biblioteka");
    gradska.DodajClana(amina);
    gradska.DodajClana(goran);
    univerzitetska.DodajClana(berun);

    try {
        //DodajClana onemogucava dodavanje clana sa istim clanskim
brojem i baca izuzetak
```

```

        gradska.DodajClana(amina);
    }
    catch (exception& e) {
        cout << "Exception: " << e.what() << crt;
    }

    //EvidentirajPosudbu pronalazi clana na osnovu clanskog broja i
    dodaje mu posudbu.
    //i dalje vase pravila definisana u metodi DodajPosudbu. metoda
    vraća true ili false.
    if (gradska.EvidentirajPosudbu(goran.GetClanskiBroj(), pjesme))
        cout << "Posudba evidentirana" << crt;

    //AktivniClanovi vraća pokazivace na clanove koji imaju najmanje
    onoliko posudbi
    //koliko je definisano vrijednoscu prosljedjenog parametra.
    vector<ClanBiblioteke*> aktivni = gradska.AktivniClanovi(1);
    for (auto clan : aktivni)
        cout << clan->GetImePrezime() << " ima " << clan-
>GetPosudbe().size()
        << " aktivnih posudbi" << crt;

    //PosudbePoZanru vraća kolekciju parova (clan, broj posudbi) za
    sve clanove koji imaju
    //najmanje jednu aktivnu posudbu knjige prosljedjenog zanra.
    Kolekcija<ClanBiblioteke, int, 50> strucnePosudbe =
gradska.PosudbePoZanru(STRUCNA_LITERATURA);
    for (int i = 0; i < strucnePosudbe.GetTrenutno(); i++)
        cout << strucnePosudbe.GetPrvi(i).GetImePrezime() << " -> "
        << strucnePosudbe.GetDrugi(i) << " posudbi" << crt;

    vector<Biblioteka> biblioteke;
    biblioteke.push_back(gradska);
    biblioteke.push_back(univerzitetska);

    /*
    Funkcija UcitajPodatke ucitava podatke o bibliotekama i njihovim
    clanovima iz datoteke
    cije ime se prosljedjuje kao prvi parametar. Svaka linija je
    zapisana u formatu:

        naziv biblioteke|ime i prezime clana

    Za svaki ispravan red potrebno je:
    - pronaci postojecu ili kreirati novu biblioteku,
    - kreirati i dodati clana u odgovarajucu biblioteku,
    - onemoguciti dupliranje biblioteka i clanova unutar iste
    biblioteke.

    Funkcija vraća true ako je ucitan najmanje jedan novi podatak, a
    false ako datoteka ne
    postoji ili nije ucitan nijedan novi podatak.

    Primjer sadržaja datoteke:
        Gradska biblioteka|Emina Junuz
        Gradska biblioteka|Jasmin Azemovic
        Univerzitetska biblioteka|Zanin Vejzovic
    */

```

```
if (UcitajPodatke("clanovi.txt", biblioteke))
    cout << "Ucitavanje uspjesno" << crt;
for (auto& biblioteka : biblioteke)
    cout << biblioteka.GetNaziv() << " sa " <<
biblioteka.GetClanovi().size() << " clanova" << crt;

    cin.get();
    return 0;
}
```